```
!git clone https://github.com/mohammad1774/Introduction_deep_RL_ELG5214.git
```

```
Cloning into 'Introduction_deep_RL_ELG5214'...
remote: Enumerating objects: 564, done.
remote: Counting objects: 100% (24/24), done.
remote: Compressing objects: 100% (18/18), done.
remote: Total 564 (delta 6), reused 18 (delta 6), pack-reused 540 (from 1)
Receiving objects: 100% (564/564), 201.91 MiB | 22.49 MiB/s, done.
Resolving deltas: 100% (155/155), done.
```

```
import os
os.chdir("Introduction_deep_RL_ELG5214/asst2")
```

```
!pip install -r requirements.txt
```

Show hidden output

```
!python run_reinforce.py
```

```
  REINFORCE seed=4  gamma=0.99  lr=0.1: 7m 32.88s
```

```
 REINFORCE seed=4, gamma=0.99, lr=0.01 ...
########################################
 [23/60] REINFORCE  seed=4, gamma=0.99, lr=0.01
Episode 50/500 - Avg Reward: -2.39, Avg Length: 36.08, Avg Loss: 0.0049
Episode 100/500 - Avg Reward: -0.16, Avg Length: 32.44, Avg Loss: 0.3869
Episode 150/500 - Avg Reward: 1.43, Avg Length: 29.60, Avg Loss: 0.6303
Episode 200/500 - Avg Reward: -0.48, Avg Length: 34.58, Avg Loss: 0.2216
Episode 250/500 - Avg Reward: 2.51, Avg Length: 28.36, Avg Loss: 0.7371
Episode 300/500 - Avg Reward: 4.32, Avg Length: 23.44, Avg Loss: 1.0033
Episode 350/500 - Avg Reward: 7.03, Avg Length: 17.52, Avg Loss: 1.3481
Episode 400/500 - Avg Reward: 6.92, Avg Length: 16.54, Avg Loss: 1.3103
Episode 450/500 - Avg Reward: 7.53, Avg Length: 14.58, Avg Loss: 1.3330
```

```
pip install gymnax
```

Show hidden output

```
"""
kaggle_zip_and_email.py  —  Zip your output folder and email it to yourself.

Paste into a Kaggle notebook cell. Uses Gmail SMTP (no extra installs needed).

SETUP (one-time):
  1. Go to https://myaccount.google.com/apppasswords
  2. Generate an App Password for "Mail"
  3. Paste it below (NOT your real Gmail password)

If you have 2FA disabled, you'll need to enable it first to get app passwords.
Alternatively, use Kaggle Secrets to store the password securely.
"""

import os
import zipfile
import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.base import MIMEBase
from email.mime.text import MIMEText
from email import encoders

# ─────────────────────────────────────────────
#  CONFIG — fill these in
# ─────────────────────────────────────────────

SENDER_EMAIL = "mohammad.ahmed1774@gmail.com"          # Your Gmail address
APP_PASSWORD  = "muoe kxnj eocf erov"           # Gmail App Password (NOT your login password)
RECIPIENT     = "mohammad.ahmed1774@gmail.com"          # Where to send (can be the same)

# Folder to zip
SOURCE_FOLDER = "Introduction_deep_RL_ELG5214/asst2"

# Zip output path
ZIP_PATH = "assignment2_output.zip"

# What to include (None = everything)
# INCLUDE_ONLY = ["metrics", "checkpoints", "plots", "logs"]
INCLUDE_ONLY = None

# Max attachment size Gmail allows (25 MB)
MAX_SIZE_MB = 24


# ─────────────────────────────────────────────
#  STEP 1: Zip the folder
# ─────────────────────────────────────────────
```

```
def zip_folder(source, output_zip, include_only=None):
    if not os.path.exists(source):
        print(f"ERROR: {source} not found")
        for item in os.listdir("/"):
            print(f"{item}")
        return False

    count = 0
    with zipfile.ZipFile(output_zip, "w", zipfile.ZIP_DEFLATED) as zf:
        for root, dirs, files in os.walk(source):
            rel_root = os.path.relpath(root, source)
            if include_only and rel_root == ".":
                dirs[:] = [d for d in dirs if d in include_only]
            for f in files:
                fp = os.path.join(root, f)
                arcname = os.path.join(os.path.basename(source),
                                       os.path.relpath(fp, source))
                zf.write(fp, arcname)
                count += 1

    size_mb = os.path.getsize(output_zip) / (1024 * 1024)
    print(f"Zipped {count} files -> {output_zip} ({size_mb:.1f} MB)")
    return size_mb


# ─────────────────────────────────────────────
#  STEP 2: Email the zip
# ─────────────────────────────────────────────

def send_email(sender, password, recipient, zip_path):
    size_mb = os.path.getsize(zip_path) / (1024 * 1024)
    if size_mb > MAX_SIZE_MB:
        print(f"ERROR: Zip is {size_mb:.1f} MB — exceeds Gmail's 25 MB limit.")
        print("Use INCLUDE_ONLY to reduce size, or use Google Drive instead.")
        return False

    msg = MIMEMultipart()
    msg["From"] = sender
    msg["To"] = recipient
    msg["Subject"] = "Assignment 2 — RL Experiment Results"

    body = (
        "Assignment 2 output attached.\n\n"
        "Contents:\n"
        "  - metrics/     Episode + summary CSVs\n"
        "  - checkpoints/ Trained model weights (.npz)\n"
        "  - plots/       Learning curves, heatmaps\n"
        "  - logs/        Training logs\n"
    )
    msg.attach(MIMEText(body, "plain"))

    # Attach the zip file
    filename = os.path.basename(zip_path)
    with open(zip_path, "rb") as f:
        part = MIMEBase("application", "zip")
        part.set_payload(f.read())

    encoders.encode_base64(part)
    part.add_header("Content-Disposition", f"attachment; filename={filename}")
    msg.attach(part)

    # Send via Gmail SMTP
    print(f"Sending {size_mb:.1f} MB to {recipient}...")
```

```
    try:
        with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
            server.login(sender, password)
            server.send_message(msg)
        print("Sent successfully!")
        return True
    except smtplib.SMTPAuthenticationError:
        print("ERROR: Authentication failed.")
        print("Make sure you're using a Gmail App Password, not your login password.")
        print("Generate one at: https://myaccount.google.com/apppasswords")
        return False
    except Exception as e:
        print(f"ERROR: {e}")
        return False


# ─────────────────────────────────────────────
#  RUN
# ─────────────────────────────────────────────

print("Step 1: Zipping folder...")
size = zip_folder(SOURCE_FOLDER, ZIP_PATH, INCLUDE_ONLY)

if size:
    print(f"\nStep 2: Emailing to {RECIPIENT}...")
    send_email(SENDER_EMAIL, APP_PASSWORD, RECIPIENT, ZIP_PATH)
```

```
Step 1: Zipping folder...
ERROR: Introduction_deep_RL_ELG5214/asst2 not found
dev
lib32
sbin
libx32
mnt
lib64
opt
etc
proc
usr
run
home
media
sys
boot
lib
root
var
srv
tmp
bin
content
kaggle
.dockerenv
tools
datalab
python-apt
python-apt.tar.xz
NGC-DL-CONTAINER-LICENSE
cuda-keyring_1.1-1_all.deb
```